EVIDENCIA DE BASE DE DATOS

Proyecto Entre Platos y Copas

Persistencia, modelo de datos y trazabilidad

Fase	Fase 2
Integrantes	Helen Cortez y Benjamín Romero
Sección	002D
Motor evaluado	PostgreSQL 17.6 en Supabase
Aplicación	https://epyc-in7j.onrender.com

1. Arquitectura de persistencia

La aplicación utiliza el ORM de Django para persistir clientes, cotizaciones, reservas, artículos y movimientos operativos. Las reglas de negocio se ejecutan en la aplicación y sus resultados quedan almacenados en la base de datos para mantener trazabilidad y consistencia.

```
Interfaz web (sitio público y panel administrativo)
  ↓
Vistas y servicios Django (validaciones, transacciones, reglas de negocio)
  ↓
Django ORM (modelos de la app Inventario)
  ↓
Base de datos PostgreSQL 17.6 · Supabase
```

Motor y método de obtención

Versión evaluada	Aplicación publicada en Render (https://epyc-in7j.onrender.com) conectada a PostgreSQL 17.6 en Supabase.
Entorno local	Para desarrollo el proyecto puede usar SQLite (variable DB_ENGINE). Ninguna evidencia de este documento proviene de SQLite.
Obtención	Consultas SQL de solo lectura ejecutadas sobre la base de producción el 05/11/2026, filtradas por el caso de prueba. No se modificaron datos ni se exportó la base.
Caso de prueba	El mismo de la evidencia funcional: COT-20261105-646A4D, clienta ficticia Camila Fuentes.

2. Modelo entidad-relación

El diagrama se generó a partir de los modelos Django desplegados y muestra las 17 tablas del dominio más la tabla intermedia servicio_categoria_solicitada, con sus llaves primarias (PK), llaves foráneas (FK, indicando la tabla referenciada) y los campos principales. Las tablas se agrupan por módulo (Comercial, Operación, Catálogo e inventario, Trazabilidad, y Acceso y empresa) y siguen el orden del ciclo de arriendo de izquierda a derecha. Solo incluye entidades que existen en el código y en la base.

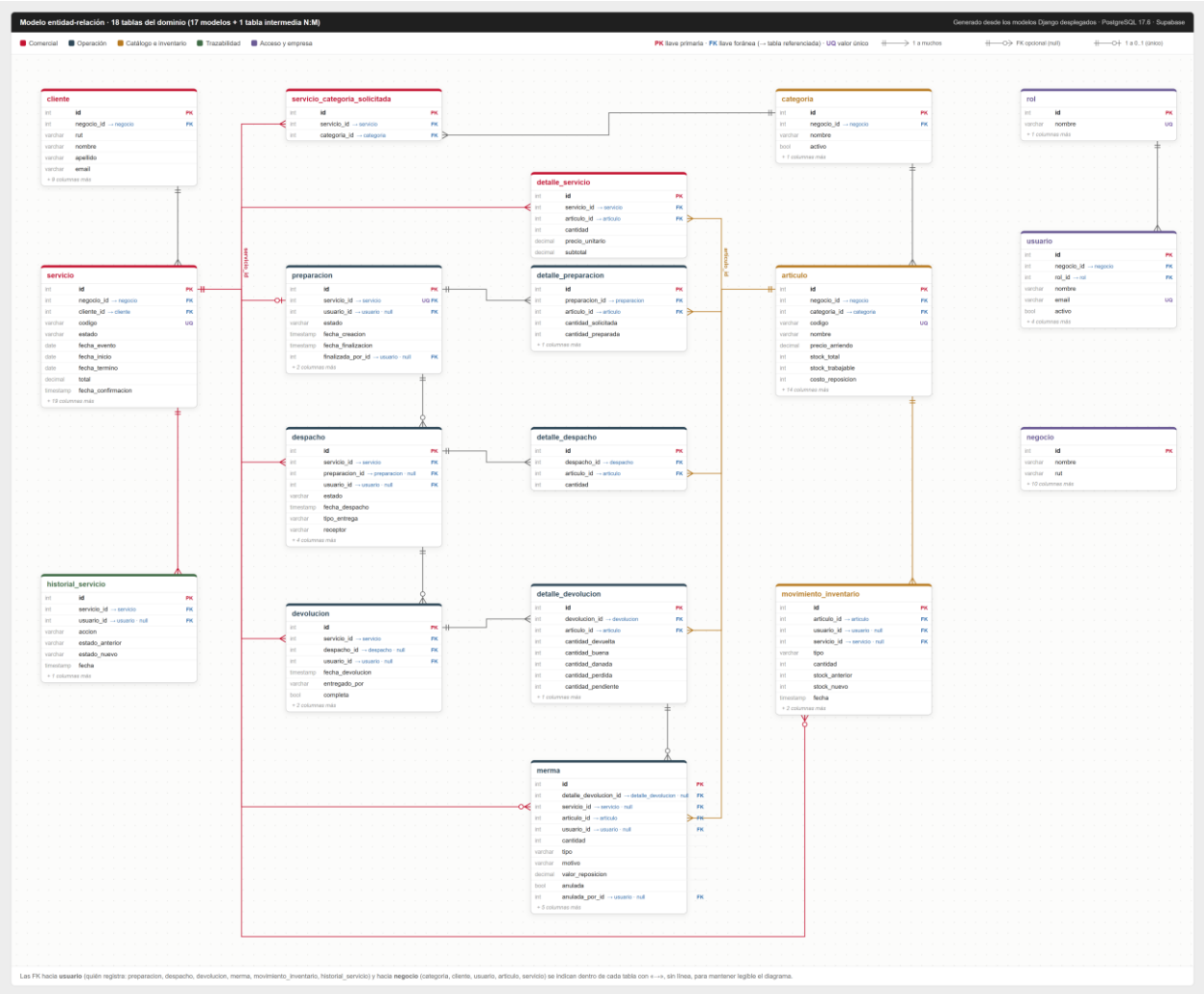


Figura 1. Modelo entidad-relación actualizado (01_Modelo_Entidad_Relacion.png)

Para no saturar el diagrama, las llaves foráneas hacia usuario (quién registró la operación) y hacia negocio aparecen como FK dentro de cada tabla, sin línea de relación.

Validación contra el esquema real

El diagrama se contrastó con el catálogo de PostgreSQL (information_schema y pg_constraint) mediante consultas de solo lectura. Resultado: sin diferencias.

Comprobación	Resultado
Tablas del dominio	18 en la base y 18 en el diagrama
Columnas por tabla	Coinciden nombres y cantidad (las no mostradas se indican como «+ N columnas más»)
Tipos de datos de las columnas mostradas	Coinciden
Llaves foráneas	37 en la base (35 de las 17 tablas del dominio y 2 de la tabla intermedia): 24 dibujadas como línea y 13 hacia usuario o negocio indicadas dentro de cada tabla
Columnas opcionales (null)	Coinciden con la nulabilidad de la base
Valores únicos (UQ)	5 columnas marcadas, iguales a las restricciones UNIQUE de la base
Cardinalidad servicio → preparación	1 a 0..1: la base impide más de una preparación por servicio (preparacion_unica_por_servicio)

3. Descripción de entidades y relaciones

Todas las tablas usan id como llave primaria autoincremental.

Tabla	Propósito	Relaciones principales
negocio	Datos de la empresa y porcentajes de abono y garantía.	Agrupa clientes, categorías, artículos, usuarios y servicios.
categoria	Clasificación de los artículos del catálogo.	negocio (FK); tiene muchos artículos.
articulo	Producto arrendable con precio y stock total y trabajable.	categoria (FK), negocio (FK).
cliente	Solicitante: RUT, nombre, contacto y cuenta web.	negocio (FK); tiene muchos servicios.
rol	Perfil de acceso al panel (por ejemplo, Administrador).	Tiene muchos usuarios.
usuario	Persona del equipo que opera el panel.	rol (FK), negocio (FK); registra operaciones.
servicio	Cotización y, al confirmarse, reserva. Guarda estado, fechas y totales.	cliente (FK); categorías solicitadas (N:M).
servicio_categoria_solicitada	Tabla intermedia de las categorías pedidas en el cotizador.	servicio (FK) y categoria (FK).
detalle_servicio	Artículos, cantidades y precios de cada servicio.	servicio (FK), articulo (FK).
preparacion	Orden de armado de una reserva confirmada.	servicio (FK, única), usuario (FK).
detalle_preparacion	Cantidad solicitada y preparada por artículo.	preparacion (FK), articulo (FK).
despacho	Salida de la reserva al cliente.	servicio (FK), preparacion (FK), usuario (FK).
detalle_despacho	Unidades enviadas por artículo.	despacho (FK), articulo (FK).
devolucion	Recepción de lo devuelto por el cliente.	servicio (FK), despacho (FK), usuario (FK).
detalle_devolucion	Unidades en buen estado, dañadas, perdidas y pendientes.	devolucion (FK), articulo (FK).
merma	Unidades dadas de baja con su valor de reposición.	articulo, servicio, detalle_devolucion y usuario (FK).
movimiento_inventario	Bitácora de entradas, salidas y bajas de stock.	articulo, servicio y usuario (FK).
historial_servicio	Trazabilidad: acción, estado anterior, estado nuevo, usuario y fecha.	servicio (FK), usuario (FK).

Entidades auxiliares

Respecto de la vista lógica anterior, el modelo evolucionó e incorporó entidades auxiliares. No son errores de la base: dan soporte a funciones transversales del sistema.

Entidad	Función
negocio	Configuración multiempresa: datos de la empresa y porcentajes de abono y garantía; clientes, artículos, usuarios y servicios pertenecen a un negocio.
rol	Control de acceso: define el perfil de cada usuario del panel.
historial_servicio	Trazabilidad: registra cada cambio de estado del servicio con usuario, fecha y observación.
movimiento_inventario	Auditoría del inventario: bitácora de confirmaciones, salidas, devoluciones y mermas con stock anterior y nuevo.
servicio_categoria_solicitada	Categorías solicitadas por el cliente en el cotizador (relación N:M entre servicio y categoría).

4. Lista de tablas reales

El esquema public contiene 28 tablas: las 17 del dominio, la tabla intermedia servicio_categoria_solicitada y 10 tablas internas de Django (auth_*, django_*). La consulta usa information_schema y no muestra cadenas de conexión ni credenciales.

Tablas del esquema public — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select table_name as tabla,
       (select count(*) from information_schema.columns c
        where c.table_schema = t.table_schema and c.table_name = t.table_name) as columnas
from information_schema.tables t
where table_schema = 'public' and table_type = 'BASE TABLE'
order by table_name;
```

tabla	columnas
articulo	23
auth_group	2
auth_group_permissions	3
auth_permission	4
auth_user	11
auth_user_groups	3
auth_user_user_permissions	3
categoria	5
cliente	15
despacho	12
detalle_despacho	4
detalle_devolucion	9
detalle_preparacion	6
detalle_servicio	6
devolucion	9
django_admin_log	8
django_content_type	3
django_migrations	4
django_session	3
historial_servicio	8
merma	16
movimiento_inventario	11
negocio	13
preparacion	9
rol	3
servicio	29
servicio_categoria_solicitada	3
usuario	10

28 filas

Figura 2. Tablas del esquema public (02_Lista_Tablas.png)

5. Persistencia del caso de prueba

Relación de los registros del caso en la base:

```
cliente  id 10 · Camila Fuentes · RUT 165483928
  └── servicio  id 20 · COT-20261105-646A4D · Cerrado · total $37.500
      ├── detalle_servicio  id 24 · Vaso old fashioned · 30 × $450 = $13.500
      ├── detalle_servicio  id 25 · Plato base negro mate · 30 × $800 = $24.000
      ├── preparacion  id 1 · Completa · 2 detalles · 60 unidades
      ├── despacho  id 1 · Entregado · 2 detalles · 60 unidades
      ├── devolucion  id 1 · completa · 57 en buen estado · 3 registradas como perdidas
      ├── merma  id 1 · 3 Vaso old fashioned · tipo Perdido · $1.350
      ├── historial_servicio  9 registros
      └── movimiento_inventario  7 registros
```

Cliente y cotización persistidos

La solicitud enviada desde el cotizador quedó guardada con su cliente, código, fecha del evento, invitados, estado y totales. Los dos detalles registran 30 unidades de cada producto con subtotales de $13.500 y $24.000.

Cliente del caso de prueba (datos ficticios) — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select id, nombre, apellido, rut, comuna, activo,
       to_char(fecha_creacion at time zone 'America/Santiago', 'DD/MM/YYYY HH24:MI') as fecha_creacion
from cliente
where rut = '165483928';
```

id	nombre	apellido	rut	comuna	activo	fecha_creacion
10	Camila	Fuentes	165483928	Providencia	True	04/11/2026 23:34

1 fila

Figura 3. Cliente del caso de prueba (03_Cliente_Prueba.png)

Servicio (cotización / reserva) del caso — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select s.id, s.codigo, s.cliente_id, c.nombre || ' ' || c.apellido as cliente,
       s.tipo_servicio, s.fecha_evento, s.cantidad_personas as invitados,
       s.subtotal, s.total, s.estado,
       to_char(s.fecha_creacion at time zone 'America/Santiago', 'DD/MM/YYYY HH24:MI') as creada
from servicio s
join cliente c on c.id = s.cliente_id
where s.codigo = 'COT-20261105-646A4D';
```

id	codigo	cliente_id	cliente	tipo_servicio	fecha_evento	invitados	subtotal	total	estado	creada
20	COT-20261105-646A4D	10	Camila Fuentes	Cena / Banquete	2026-11-05	30	37500	37500	Cerrado	05/11/2026 00:09

1 fila

Figura 4. Servicio (cotización / reserva) persistido (04_Cotizacion_Persistida.png)

Detalle del servicio: artículos, cantidades y subtotales — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select d.id, d.servicio_id, d.articulo_id, a.codigo, a.nombre as articulo,
       d.cantidad, d.precio_unitario, d.subtotal
from detalle_servicio d
join articulo a on a.id = d.articulo_id
join servicio s on s.id = d.servicio_id
where s.codigo = 'COT-20261105-646A4D'
order by d.id;
```

id	servicio_id	articulo_id	codigo	articulo	cantidad	precio_unitario	subtotal
24	20	63	cris-004	Vaso old fashioned	30	450	13500
25	20	59	vaj-004	Plato base negro mate	30	800	24000

2 filas

Figura 5. Detalles de la cotización (05_Detalles_Cotizacion.png)

6. Reserva e inventario comprometido

Al confirmar, el servicio guarda la fecha de confirmación y el periodo que ocupa (desde/hasta), y se registra un movimiento de inventario «Reserva confirmada» por cada artículo. El stock comprometido no se guarda como un número fijo: se calcula sumando los detalles de los servicios en estados que retienen stock (Confirmado, Preparación, Preparado, Despachado y Devolución parcial) cuyo periodo se cruza con la fecha consultada.



Confirmación de la reserva: fecha, periodo y movimientos generados — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select s.codigo, s.fecha_inicio as desde, s.fecha_termino as hasta,
       to_char(s.fecha_confirmacion at time zone 'America/Santiago', 'DD/MM/YYYY HH24:MI') as fecha_confirmacion,
       m.tipo as movimiento, a.nombre as articulo, m.cantidad
from servicio s
join movimiento_inventario m on m.servicio_id = s.id and m.tipo = 'Reserva confirmada'
join articulo a on a.id = m.articulo_id
where s.codigo = 'COT-20261105-646A4D'
order by m.id;
```

codigo	desde	hasta	fecha_confirmacion	movimiento	articulo	cantidad
COT-20261105-646A4D	2026-11-05	2026-11-05	05/11/2026 00:12	Reserva confirmada	Plato base negro mate	30
COT-20261105-646A4D	2026-11-05	2026-11-05	05/11/2026 00:12	Reserva confirmada	Vaso old fashioned	30

2 filas

Figura 6. Confirmación de la reserva y movimientos generados (06_Reserva_Confirmada.png)

Variación observada en la interfaz al confirmar (Evidencia funcional 9 y 11):

Indicador	Antes de confirmar	Después de confirmar
Stock total	200	200
Disponible	120	**60**
Comprometido	80	**140**
Cantidad nueva comprometida	0	**60**

Debido a que el caso COT-20261105-646A4D ya se encuentra cerrado, dejó de comprometer inventario. Las 80 unidades todavía comprometidas corresponden a otra reserva vigente (COT-20261015-3824FF): 40 platos y 40 vasos. Por eso el estado actual es: stock total 197, disponible 117 y comprometido 80. El stock del Vaso old fashioned quedó en 97 por la merma del caso.



Unidades retenidas por reservas vigentes (estado actual) y stock de los artículos del caso — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select s.codigo, s.estado, a.nombre as articulo, sum(d.cantidad) as unidades_comprometidas,
       a.stock_total, a.stock_trabajable
from detalle_servicio d
join servicio s on s.id = d.servicio_id
join articulo a on a.id = d.articulo_id
where s.estado in ('Confirmado','Preparacion','Preparado','Despachado','Devolucion parcial')
group by s.codigo, s.estado, a.nombre, a.stock_total, a.stock_trabajable
order by s.codigo, a.nombre;
```

codigo	estado	articulo	unidades_comprometidas	stock_total	stock_trabajable
COT-20261015-3824FF	Confirmado	Plato base negro mate	40	100	100
COT-20261015-3824FF	Confirmado	Vaso old fashioned	40	97	97

2 filas

Figura 7. Unidades retenidas por reservas vigentes (07_Inventario_Comprometido.png)

7. Historial de estados

La tabla historial_servicio registra cada acción con su estado anterior, estado nuevo, fecha, hora y usuario responsable. La creación de la solicitud queda en servicio.fecha_creacion (05/11/2026 00:09).

Cotización → Contactado → Confirmado → Preparación → Preparado → Despachado → Finalizado → Cerrado

La base de datos almacena internamente el estado «Finalizado», que en la interfaz se presenta al usuario como «Devuelto». Posteriormente, la reserva pasa al estado «Cerrado».

Historial de estados del servicio — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select to_char(h.fecha at time zone 'America/Santiago', 'DD/MM/YYYY HH24:MI') as fecha, h.accion, h.estado_anterior, h.estado_nuevo,
       u.nombre || ' ' || u.apellido as usuario, h.observacion
from historial_servicio h
join servicio s on s.id = h.servicio_id
left join usuario u on u.id = h.usuario_id
where s.codigo = 'COT-20261105-646A4D'
order by h.fecha, h.id;
```

fecha	accion	estado_anterior	estado_nuevo	usuario	observacion
05/11/2026 00:12	Cotización enviada por WhatsApp	Cotizacion	Contactado	Helen Cortez	
05/11/2026 00:12	Confirmar reserva	Contactado	Confirmado	Helen Cortez	
05/11/2026 00:13	Generar preparación	Confirmado	Preparacion	Helen Cortez	2 artículos, 60 unidades por preparar.
05/11/2026 00:13	Guardar avance de preparación	Preparacion	Preparacion	Helen Cortez	0 de 60 unidades preparadas (0 %).
05/11/2026 00:13	Guardar avance de preparación	Preparacion	Preparacion	Helen Cortez	60 de 60 unidades preparadas (100 %).
05/11/2026 00:13	Finalizar preparación	Preparacion	Preparado	Helen Cortez	60 unidades preparadas y revisadas.
05/11/2026 00:14	Registrar despacho	Preparado	Despachado	Helen Cortez	60 unidades de 2 artículos. Entrega: Helen Cortez. Recibe: Camila Fuentes. Devolución: 05/11/2026.
05/11/2026 00:15	Registrar devolución	Despachado	Finalizado	Helen Cortez	57 unidades recibidas en buen estado, 3 como merma, 0 pendientes. Entregó: Camila Fuentes.
05/11/2026 00:16	Cerrar reserva	Finalizado	Cerrado	Helen Cortez	Todas las unidades quedaron resueltas.

9 filas

Figura 8. Historial de estados del servicio (08_Historial_Estados.png)

8. Preparación y despacho persistidos

La preparación quedó Completa, con responsable, fecha de creación y de finalización, y 30 de 30 unidades preparadas por artículo. El despacho quedó Entregado, asociado a esa preparación, con fecha de salida, responsable, receptor y 60 unidades.

Preparación y sus detalles — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select p.id as preparacion_id, p.estado, u.nombre || ' ' || u.apellido as responsable,
       to_char(p.fecha_creacion at time zone 'America/Santiago', 'DD/MM/YYYY HH24:MI') as creada, to_char(p.fecha_finalizacion at time zone 'America/Santiago',
'DD/MM/YYYY HH24:MI') as finalizada,
       a.nombre as articulo, dp.cantidad_solicitada, dp.cantidad_preparada
from preparacion p
join servicio s on s.id = p.servicio_id
left join usuario u on u.id = p.usuario_id
join detalle_preparacion dp on dp.preparacion_id = p.id
join articulo a on a.id = dp.articulo_id
where s.codigo = 'COT-20261105-646A4D'
order by dp.id;
```

preparacion_id	estado	responsable	creada	finalizada	articulo	cantidad_solicitada	cantidad_preparada
1	Completa	Helen Cortez	05/11/2026 00:13	05/11/2026 00:13	Plato base negro mate	30	30
1	Completa	Helen Cortez	05/11/2026 00:13	05/11/2026 00:13	Vaso old fashioned	30	30

2 filas

Figura 9. Preparación y sus detalles (09_Preparacion.png)

Despacho y sus detalles — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select d.id as despacho_id, d.preparacion_id, d.estado, d.tipo_entrega,
       to_char(d.fecha_despacho at time zone 'America/Santiago', 'DD/MM/YYYY HH24:MI') as salida, d.responsable_entrega, d.receptor,
       a.nombre as articulo, dd.cantidad
from despacho d
join servicio s on s.id = d.servicio_id
join detalle_despacho dd on dd.despacho_id = d.id
join articulo a on a.id = dd.articulo_id
where s.codigo = 'COT-20261105-646A4D'
order by dd.id;
```

despacho_id	preparacion_id	estado	tipo_entrega	salida	responsable_entrega	receptor	articulo	cantidad
1	1	Entregado	Despacho	05/11/2026 00:14	Helen Cortez	Camila Fuentes	Plato base negro mate	30
1	1	Entregado	Despacho	05/11/2026 00:14	Helen Cortez	Camila Fuentes	Vaso old fashioned	30

2 filas

Figura 10. Despacho y sus detalles (10_Despacho.png)

9. Devolución y merma persistidas

Despachadas	Buen estado	Merma	Pendientes
60	57	3	0

De los 30 vasos despachados, se devolvieron 27 en buen estado y se registraron 3 unidades como perdidas. En la base: cantidad_devuelta = 27, cantidad_buena = 27, cantidad_danada = 0, cantidad_perdida = 3 y merma de tipo «Perdido». Esto se debe a que el formulario de devolución usa una sola columna «Merma» y la registra como baja definitiva (pérdida), aunque la observación ingresada en la prueba menciona vasos quebrados. La devolución quedó marcada como completa y la merma, vigente, con motivo «Devolución de la reserva COT-20261105-646A4D». El movimiento de inventario asociado registra el stock del artículo antes (100) y después (97).

Devolución y sus detalles — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select dv.id as devolucion_id, dv.despacho_id, to_char(dv.fecha_devolucion at time zone 'America/Santiago', 'DD/MM/YYYY HH24:MI') as recepcion,
       dv.completa, a.nombre as articulo, dd.cantidad_devuelta, dd.cantidad_buena,
       dd.cantidad_danada, dd.cantidad_perdida, dd.cantidad_pendiente
from devolucion dv
join servicio s on s.id = dv.servicio_id
join detalle_devolucion dd on dd.devolucion_id = dv.id
join articulo a on a.id = dd.articulo_id
where s.codigo = 'COT-20261105-646A4D'
order by dd.id;
```

devolucion_id	despacho_id	recepcion	completa	articulo	cantidad_devuelta	cantidad_buena	cantidad_danada	cantidad_perdida	cantidad_pendiente
1	1	05/11/2026 00:15	True	Plato base negro mate	30	30	0	0	0
1	1	05/11/2026 00:15	True	Vaso old fashioned	27	27	0	3	0

2 filas

Figura 11. Devolución y sus detalles (11_Devolucion.png)

Merma registrada, valorización y movimiento de inventario asociado — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select m.id as merma_id, m.detalle_devolucion_id, a.nombre as articulo, m.cantidad, m.tipo,
       case when m.anulada then 'Anulada' else 'Vigente' end as estado,
       a.costo_reposicion, a.precio_arriendo,
       coalesce(nullif(a.costo_reposicion, 0), a.precio_arriendo) as valor_unitario_aplicado,
       m.valor_reposicion,
       case when m.valor_reposicion = m.cantidad * coalesce(nullif(a.costo_reposicion, 0), a.precio_arriendo)
            then 'Cumple' else 'No cumple' end as comprobacion,
       mi.stock_anterior, mi.stock_nuevo
from merma m
join servicio s on s.id = m.servicio_id
join articulo a on a.id = m.articulo_id
left join movimiento_inventario mi
       on mi.servicio_id = s.id and mi.tipo = 'Merma' and mi.articulo_id = m.articulo_id
where s.codigo = 'COT-20261105-646A4D';
```

merma_id	detalle_devolucion_id	articulo	cantidad	tipo	estado	costo_reposicion	precio_arriendo	valor_unitario_aplicado	valor_reposicion	comprobacion	stock_anterior	stock_nuevo
1	2	Vaso old fashioned	3	Perdido	Vigente	0	450	450	1350	Cumple	100	97

1 fila

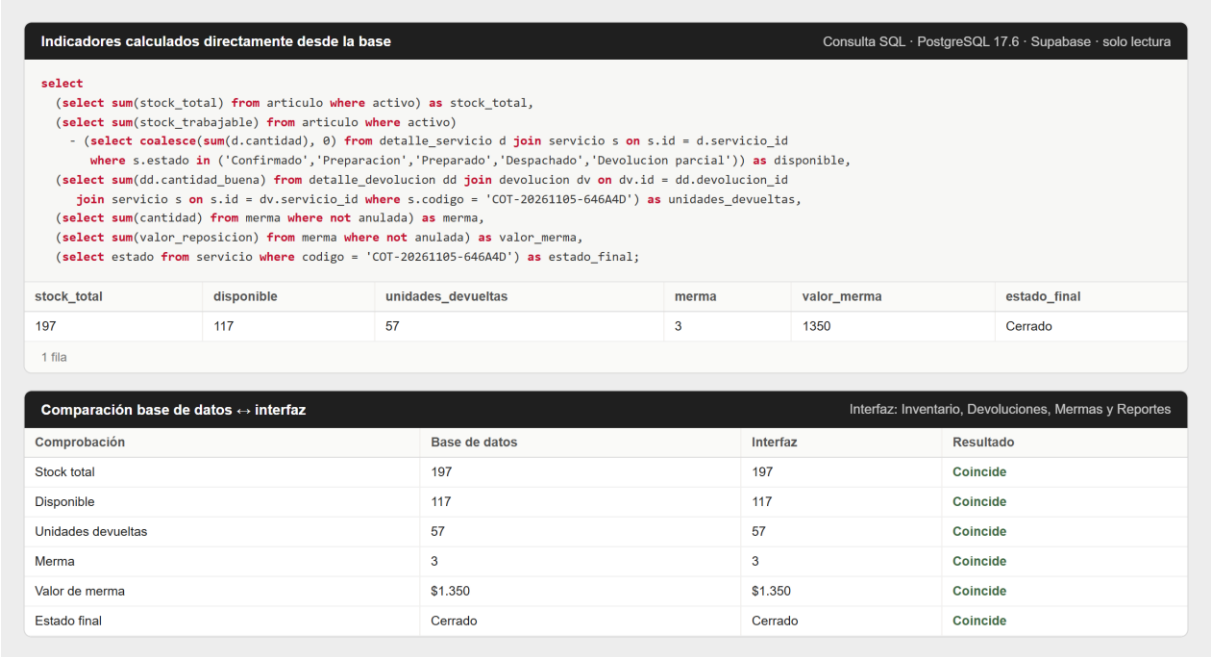
Figura 12. Merma registrada (12_Merma.png)

Valorización: el sistema usa el campo articulo.costo_reposicion cuando está cargado y, si vale 0, el precio del producto (regla valor_unitario_merma del modelo). El Vaso old fashioned tiene costo_reposicion = 0, por lo que se aplicó el precio de $450: 3 × $450 = $1.350. La consulta muestra ambos campos y comprueba que valor_reposicion = cantidad × valor unitario aplicado (Cumple). Para valorizar con el costo real basta cargar el costo de reposición del producto.

10. Consistencia entre base de datos e interfaz

Los indicadores se calcularon directamente con SQL y se compararon con los valores mostrados en la aplicación (Inventario, Devoluciones, Mermas y Reportes). Todos coinciden: las pantallas reflejan datos persistidos y no valores aislados.

Comprobación	Base de datos	Interfaz	Resultado
Stock total	197	197	Coincide
Disponible	117	117	Coincide
Unidades devueltas	57	57	Coincide
Merma	3	3	Coincide
Valor de merma	$1.350	$1.350	Coincide
Estado final	Cerrado	Cerrado	Coincide



Indicadores calculados directamente desde la base — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select
  (select sum(stock_total) from articulo where activo) as stock_total,
  (select sum(stock_trabajable) from articulo where activo)
    - (select coalesce(sum(d.cantidad), 0) from detalle_servicio d join servicio s on s.id = d.servicio_id
       where s.estado in ('Confirmado','Preparacion','Preparado','Despachado','Devolucion parcial')) as disponible,
  (select sum(dd.cantidad_buena) from detalle_devolucion dd join devolucion dv on dv.id = dd.devolucion_id
     join servicio s on s.id = dv.servicio_id where s.codigo = 'COT-20261105-646A4D') as unidades_devueltas,
  (select sum(cantidad) from merma where not anulada) as merma,
  (select sum(valor_reposicion) from merma where not anulada) as valor_merma,
  (select estado from servicio where codigo = 'COT-20261105-646A4D') as estado_final;
```

stock_total	disponible	unidades_devueltas	merma	valor_merma	estado_final
197	117	57	3	1350	Cerrado

1 fila

Comparación base de datos ↔ interfaz — Interfaz: Inventario, Devoluciones, Mermas y Reportes

Comprobación	Base de datos	Interfaz	Resultado
Stock total	197	197	Coincide
Disponible	117	117	Coincide
Unidades devueltas	57	57	Coincide
Merma	3	3	Coincide
Valor de merma	$1.350	$1.350	Coincide
Estado final	Cerrado	Cerrado	Coincide

Figura 13. Indicadores calculados en la base y comparación con la interfaz (13_Consistencia_Final.png)

11. Migraciones aplicadas

Las 10 migraciones de la app Inventario que existen en el repositorio desplegado están aplicadas en la base (marcadas con [X] y registradas en django_migrations), al igual que las migraciones internas de Django. No hay migraciones pendientes.

Migración	Contenido principal
0001_initial	Tablas base: negocio, categoría, artículo, cliente, rol, usuario, servicio y detalle.
0002_cotizador_y_disponibilidad	Datos del cotizador público y control de disponibilidad.
0003_preparacion_e_historial	Preparación, detalle de preparación e historial de estados.
0004_despacho_completo	Despacho y detalle de despacho.
0005_devolucion_merma_cierre	Devolución, detalle de devolución, merma y movimientos.
0006_cuenta_cliente	Cuenta web del cliente.
0007_costo_reposicion	Costo de reposición para valorizar mermas.
0008_aceptada_cliente	Aceptación de la cotización por el cliente.
0009_gestion_categorias	Gestión de categorías: índice único del nombre (sin distinguir mayúsculas) por negocio.
0010_normalizar_telefonos	Migración de datos: deja los celulares ya guardados en el formato +56 9 1234 5678.

python manage.py showmigrations — Ejecutado contra PostgreSQL 17.6 · Supabase

```
$ python manage.py showmigrations Inventario auth contenttypes sessions admin
Inventario
 [X] 0001_initial
 [X] 0002_cotizador_y_disponibilidad
 [X] 0003_preparacion_e_historial
 [X] 0004_despacho_completo
 [X] 0005_devolucion_merma_cierre
 [X] 0006_cuenta_cliente
 [X] 0007_costo_reposicion
 [X] 0008_aceptada_cliente
 [X] 0009_gestion_categorias
 [X] 0010_normalizar_telefonos
auth
 [X] 0001_initial
 [X] 0002_alter_permission_name_max_length
 [X] 0003_alter_user_email_max_length
 [X] 0004_alter_user_username_opts
 [X] 0005_alter_user_last_login_null
 [X] 0006_require_contenttypes_0002
 [X] 0007_alter_validators_add_error_messages
 [X] 0008_alter_user_username_max_length
 [X] 0009_alter_user_last_name_max_length
 [X] 0010_alter_group_name_max_length
 [X] 0011_update_proxy_permissions
 [X] 0012_alter_user_first_name_max_length
contenttypes
 [X] 0001_initial
 [X] 0002_remove_content_type_name
sessions
 [X] 0001_initial
admin
 [X] 0001_initial
 [X] 0002_logentry_remove_auto_add
 [X] 0003_logentry_add_action_flag_choices
```

Migraciones registradas en la tabla django_migrations (aplicación Inventario) — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select id, app, name as migracion, to_char(applied at time zone 'America/Santiago', 'DD/MM/YYYY HH24:MI') as aplicada
from django_migrations
where app = 'Inventario'
order by id;
```

id	app	migracion	aplicada
1	Inventario	0001_initial	12/09/2026 22:09
20	Inventario	0002_cotizador_y_disponibilidad	15/10/2026 12:13
21	Inventario	0003_preparacion_e_historial	31/10/2026 22:24
22	Inventario	0004_despacho_completo	31/10/2026 22:24
23	Inventario	0005_devolucion_merma_cierre	31/10/2026 22:25
24	Inventario	0006_cuenta_cliente	31/10/2026 22:25
25	Inventario	0007_costo_reposicion	31/10/2026 22:25
26	Inventario	0008_aceptada_cliente	31/10/2026 22:25
27	Inventario	0009_gestion_categorias	05/11/2026 11:46
28	Inventario	0010_normalizar_telefonos	05/11/2026 14:47

10 filas

Figura 14. Migraciones aplicadas en la base (14_Migraciones_Aplicadas.png)

12. Integridad y reglas de datos

En las 17 tablas del dominio la base define 35 llaves foráneas, 20 restricciones CHECK, 17 llaves primarias y 7 restricciones de unicidad; desde la migración 0009 se suma un índice único para el nombre de las categorías. Se indica en cada caso si la regla la aplica la base de datos o la capa de servicios de Django.

- **Llaves foráneas:** cada detalle, preparación, despacho, devolución y merma referencia a su servicio y artículo. Las referencias críticas usan PROTECT (no se puede borrar un cliente o artículo con operaciones) y los detalles se eliminan en cascada con su cabecera. Restricción de base de datos.
- **Cantidades no negativas:** CHECK (cantidad >= 0) en detalles, stock y merma; merma exige cantidad > 0. Restricción de base de datos.
- **Stock coherente:** CHECK (stock_trabajable <= stock_total) en artículo. Restricción de base de datos.
- **Preparación acotada:** CHECK (cantidad_preparada <= cantidad_solicitada) y una sola preparación por servicio (UNIQUE). Restricción de base de datos.
- **Devolución coherente:** CHECK (cantidad_devuelta = cantidad_buena + cantidad_danada) en la base. Que lo devuelto no supere lo despachado se valida en la capa de servicios de Django antes de persistir la operación.
- **Categorías únicas por negocio:** índice único sobre lower(nombre) y negocio_id en la tabla categoria (migración 0009): «Iluminación» e «ILUMINACIÓN» no pueden coexistir en el mismo negocio. Es un índice único, por lo que no aparece entre las restricciones de pg_constraint. Restricción de base de datos.
- **Reserva asociada a un cliente:** servicio.cliente_id es obligatorio (NOT NULL + FK).
- **Prevención de sobreventa:** al confirmar, Django abre una transacción (transaction.atomic), bloquea las filas de los artículos (SELECT … FOR UPDATE) y recalcula la disponibilidad para las fechas del servicio. Si falta stock, la operación se revierte. Se valida en la capa de servicios de Django.
- **Operaciones atómicas:** despacho, devolución, merma y cierre se ejecutan dentro de transacciones: o se guarda todo el registro o no se guarda nada.

Cantidad de restricciones por tipo (17 tablas del dominio) — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select case contype when 'p' then 'Llave primaria' when 'f' then 'Llave foránea'
                    when 'u' then 'Única' when 'c' then 'Check' end as tipo, count(*) as cantidad
from pg_constraint c
where connamespace = 'public'::regnamespace
  and conrelid::regclass::text not like 'auth%' and conrelid::regclass::text not like 'django%'
  and conrelid::regclass::text <> 'servicio_categoria_solicitada'
group by contype order by 2 desc;
```

tipo	cantidad
Llave foránea	35
Check	20
Llave primaria	17
Única	7

4 filas

Restricciones CHECK definidas en la base (tablas de inventario y operación) — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select conrelid::regclass as tabla, conname as restriccion, pg_get_constraintdef(oid) as regla
from pg_constraint
where contype = 'c' and connamespace = 'public'::regnamespace
  and conrelid::regclass::text in ('articulo', 'detalle_servicio', 'detalle_preparacion',
                                   'detalle_despacho', 'detalle_devolucion', 'merma')
order by 1, 2;
```

tabla	restriccion	regla
articulo	articulo_costo_reposicion_check	CHECK ((costo_reposicion >= 0))
articulo	articulo_stock_total_check	CHECK ((stock_total >= 0))
articulo	articulo_stock_trabajable_check	CHECK ((stock_trabajable >= 0))
articulo	stock_trabajable_no_supera_total	CHECK ((stock_trabajable <= stock_total))
detalle_despacho	detalle_despacho_cantidad_check	CHECK ((cantidad >= 0))
detalle_devolucion	detalle_devolucion_cantidad_buena_check	CHECK ((cantidad_buena >= 0))
detalle_devolucion	detalle_devolucion_cantidad_danada_check	CHECK ((cantidad_danada >= 0))
detalle_devolucion	detalle_devolucion_cantidad_devuelta_check	CHECK ((cantidad_devuelta >= 0))
detalle_devolucion	detalle_devolucion_cantidad_pendiente_check	CHECK ((cantidad_pendiente >= 0))
detalle_devolucion	detalle_devolucion_cantidad_perdida_check	CHECK ((cantidad_perdida >= 0))
detalle_devolucion	devuelta_es_buena_mas_danada	CHECK ((cantidad_devuelta = (cantidad_buena + cantidad_danada)))
detalle_preparacion	detalle_preparacion_cantidad_preparada_check	CHECK ((cantidad_preparada >= 0))
detalle_preparacion	detalle_preparacion_cantidad_solicitada_check	CHECK ((cantidad_solicitada >= 0))
detalle_preparacion	preparada_no_supera_solicitada	CHECK ((cantidad_preparada <= cantidad_solicitada))
detalle_servicio	detalle_servicio_cantidad_check	CHECK ((cantidad >= 0))
merma	merma_cantidad_check	CHECK ((cantidad >= 0))
merma	merma_cantidad_positiva	CHECK ((cantidad > 0))
merma	merma_cantidad_reparada_check	CHECK ((cantidad_reparada >= 0))
merma	merma_reparada_no_supera	CHECK ((cantidad_reparada <= cantidad))

19 filas

Índice único de la tabla categoria (nombre sin distinguir mayúsculas, por negocio) — Consulta SQL · PostgreSQL 17.6 · Supabase · solo lectura

```
select indexname as indice, indexdef as definicion
from pg_indexes
where schemaname = 'public' and tablename = 'categoria'
order by 1;
```

indice	definicion
categoria_negocio_id_c65ae6ee	CREATE INDEX categoria_negocio_id_c65ae6ee ON public.categoria USING btree (negocio_id)
categoria_nombre_unico_por_negocio	CREATE UNIQUE INDEX categoria_nombre_unico_por_negocio ON public.categoria USING btree (lower((nombre)::text), negocio_id)
categoria_pkey	CREATE UNIQUE INDEX categoria_pkey ON public.categoria USING btree (id)

3 filas

Figura 15. Restricciones de integridad definidas en la base (15_Restricciones_Integridad.png)

13. Resguardo de la información

Para esta evidencia no se incluyó ni se subió:

- La base de datos real, archivos .sqlite3 ni exportaciones de Supabase.
- Archivos .env, contraseñas, tokens, claves de API ni cadenas de conexión.
- Datos personales reales: las consultas se filtraron por el caso de prueba, cuya clienta es ficticia.
- Tablas completas con información de otros clientes.

Conclusión

Las consultas sobre la base de producción demuestran que cada etapa del ciclo de arriendo queda persistida y relacionada: la cotización con su cliente y detalles, la reserva con su historial de estados, la preparación, el despacho, la devolución y la merma. Los indicadores calculados en la base coinciden con los que muestra la aplicación, las migraciones están aplicadas y la integridad se resguarda con restricciones en la base y validaciones transaccionales en Django.